RESOLUÇÃO COMENTADA

A Caixa de Lembranças

Do bucket de lembranças ao Bedrock, com o porquê de cada decisão.

01 Antes de começar

Região. `us-east-1` para tudo, inclusive o **Model access** do Bedrock, que é habilitado **por região**. Modelo habilitado na região errada é o erro mais demorado de diagnosticar do desafio.

Orçamento. Budget de US$ 5 com alertas. Este desafio volta a caber no Free Tier (uma t3.micro) e o Bedrock cobra por token: centavos. O risco aqui não é preço, é esquecer a instância; o hábito continua.

Par de chaves. `cecilia.pem`, `chmod 400`, o ritual de sempre.

> 💡 **Antes do encontro:** o instrutor deve rodar este guia inteiro numa conta limpa. Os pontos que costumam doer: Model access demorando alguns minutos para propagar, o ID do modelo de chat variando por conta (verificar na seção 03), e contas AWS Academy/da instituição com Bedrock bloqueado por SCP. Descobrir isso na frente da turma é péssimo.

02 O bucket de lembranças (tarefa 1)

S3 → Create bucket → `cecilia-lembrancas-SEUNOME` (nome global, minúsculas). **Block all public access: LIGADO.** As memórias da vó não são página pública; quem lê o bucket é a API, autenticada pela role.

Upload dos 10 arquivos de `lembrancas/` do clone do repositório (console, ou `aws s3 sync lembrancas/ s3://cecilia-lembrancas-SEUNOME/`).

Decisão comentada: dois buckets no desafio, com posturas opostas: o de lembranças (privado, lido pela role) e o do site (público, lido por navegador). Colocar os dois lado a lado é proposital: "público ou privado" deixa de ser um checkbox e vira uma pergunta por dado.

03 Model access no Bedrock (tarefa 2)

Console → **Amazon Bedrock** → Model access (canto inferior esquerdo) → Modify model access:

- **Titan Text Embeddings V2** (Amazon), que vira os vetores da busca;
- **Claude Haiku 4.5** (Anthropic), que escreve as respostas da vó. O formulário da Anthropic pede o caso de uso: "projeto educacional" resolve.

Aguardar *Access granted* (minutos). Habilitar não custa nada; a cobrança é por token invocado.

Os IDs no código (e a pegadinha do prefixo):

- Embeddings: `amazon.titan-embed-text-v2:0`, chamado direto.
- Chat: `us.anthropic.claude-haiku-4-5-20251001-v1:0`. Repare no `us.`: é um **inference profile**, não o modelo cru. Os Claude atuais só aceitam invocação on-demand via profile, que roteia a chamada entre as regiões `us-*`. Sem o prefixo: `on-demand throughput isn't supported`. A API já traduz esse erro em PT-BR.
- **Verificar na conta** (os catálogos mudam): Bedrock → Cross-region inference lista os profiles disponíveis. Se o Haiku 4.5 não aparecer, usar `us.anthropic.claude-3-5-haiku-20241022-v1:0` e ajustar a variável `MODELO_CHAT` no `docker run`; o backend aceita override por env exatamente para isso.

04 A IAM Role e a policy mínima (tarefa 3)

O coração do desafio. IAM → Policies → Create policy → JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListarBucketDeLembrancas",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::cecilia-lembrancas-SEUNOME"
    },
    {
      "Sid": "LerLembrancas",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::cecilia-lembrancas-SEUNOME/*"
    },
    {
      "Sid": "InvocarModelos",
      "Effect": "Allow",
      "Action": "bedrock:InvokeModel",
      "Resource": [
        "arn:aws:bedrock:us-east-1::foundation-model/amazon.titan-embed-text-v2:0",
        "arn:aws:bedrock:us-east-1:NUMERO_DA_CONTA:inference-profile/us.anthropic.claude-haiku-4-5*",
        "arn:aws:bedrock:us-*::foundation-model/anthropic.claude-haiku-4-5*"
      ]
    }
  ]
}
```

As quatro decisões da policy:

1. **`ListBucket` no bucket, `GetObject` no `/*`.** Ações de S3 miram ARNs diferentes: listar é operação *do bucket*, ler é operação *do objeto*. Juntar tudo num ARN só é o erro nº 1 da tarefa (e o `/health` denuncia:

`bucket: "sem_acesso"`).

2. **O chat precisa de DOIS tipos de recurso.** A chamada entra pelo inference profile (**da conta**, por isso o `NUMERO_DA_CONTA`) e o profile a repassa a um foundation model em `us-east-1`, `us-east-2` ou `us-west-2`. Por isso o terceiro ARN é `us-*` e sem número de conta (foundation models são recursos públicos regionais). Cobrir só o profile dá `AccessDeniedException` **na segunda perna**, o erro mais instrutivo do desafio.
3. **Nenhum wildcard de ação.** `bedrock:*` ou `s3:*` funcionam e reprovam: o contrato do README lista três ações, a policy tem três.
4. **Nada de `PutObject` ... ainda.** O arremate (seção 09) vai exigi-lo, e a graça é o aluno descobrir isso pelo erro.

Depois: IAM → Roles → Create role → **AWS service / EC2** → anexar a policy → nome `cecilia-caixa-role`. (O console cria o *instance profile* homônimo sozinho; na CLI seriam dois passos, vale mencionar em aula.)

05 A EC2 com a role (tarefa 4)

EC2 → Launch instance:

- Ubuntu 24.04 LTS · `t3.micro` · key pair `cecilia`
- **Security Group**: 22 ← `SEU_IP/32` · 3000 ← `0.0.0.0/0` (o site fala direto com a API, como no Dojo; sem banco, sem 5432)
- **Advanced details → IAM instance profile:** `cecilia-caixa-role`
- **Advanced details → Metadata response hop limit:** `2`. No Desafio 02 o limite 1 escondia o crachá; aqui ele **derruba as credenciais**: o SDK dentro do container busca as credenciais da role no serviço de metadados, e a resposta morre um salto antes. Sintoma: `CredentialsProviderError` / timeout em toda chamada AWS.

Dentro (`ssh -i cecilia.pem ubuntu@IP`):

```
sudo apt-get update -y && sudo apt-get install -y docker.io

sudo docker run -d --name caixa --restart unless-stopped -p 3000:3000 \
  -e S3_BUCKET_LEMBRANCAS=cecilia-lembrancas-SEUNOME \
  -e AWS_REGION=us-east-1 \
  ghcr.io/liga-de-computacao-em-nuvem-pucrs/desafio-03-velhinha-backend:latest

curl -s localhost:3000/health
```

Esperado: `"bucket": "ok"` e `"documentosIndexados": 10` (a API indexa na subida). **Repare no que NÃO está no comando:** nenhuma credencial. É a evolução em relação ao Dojo: mesma aplicação, mesmo SDK, zero segredo. A cadeia de credenciais achou a role sozinha.

Se a instância subiu **antes** da role/hop limit: Actions → Security → Modify IAM role, e Actions → Instance settings → Modify instance metadata options → hop limit 2, seguido de `sudo docker restart caixa` (o SDK cacheia a falha de credencial por um tempo).

06 Indexar e sabatinar (tarefa 5)

```
curl -s -X POST localhost:3000/api/reindexar
# → { "indexados": 10, "trechos": ~20-25, "arquivos": [...], "duracaoMs": ... }
```

A sabatina, pelo site ou por curl, com perguntas de resposta **verificável nos arquivos**:

PERGUNTA	RESPOSTA ESPERADA	LEMBRANÇA
Quantos ovos vão no bolo de laranja?	3	receita-do-bolo-de-laranja.md
Em que ano foi a enchente?	1967 (agosto)	a-enchente-de-1967.md
Quantas cartas estão na lata?	47	cartas-do-vo.md
Qual o time de coração da vó?	**"não está na minha caixa, meu bem"**	nenhuma

A quarta linha é o teste que o enunciado chama de recusa honesta, e tem **duas** camadas no código: o limiar de similaridade (~0,35) corta o caso "nada parecido" antes de gastar token com o modelo, e o prompt manda admitir quando os trechos vieram mas não respondem. Mostrar as duas no `rag.js` rende a melhor discussão do encontro: alucinação não se resolve pedindo "por favor não invente"; se resolve **não dando material** para inventar e **medindo** o que a busca devolveu (os trechos com a similaridade aparecem na tela para exatamente isso).

07 O site no S3 (tarefa 6)

Igual ao Desafio 02: bucket público com static website hosting, e o `config.js` de `dist/` editado, desta vez com IP **e porta**:

```
window.LIGA_CONFIG = { API_URL: 'http://IP_DA_EC2:3000' };
```

```
aws s3 sync frontend/dist/ s3://cecilia-site-SEUNOME/
aws s3 cp frontend/dist/config.js s3://cecilia-site-SEUNOME/config.js \
  --cache-control no-store
```

Abrir pela URL de website (http) e deixar a turma perguntar. Os selos no topo da página são o `/health` traduzido: se algo estiver pendente, o próprio site conta o quê.

08 A estimativa de custos (tarefa 7)

A conta de tokens explicada para leigo (preços de meados de 2026, confirmar; Titan v2 ≈ US$ 0,00002/1k tokens; Haiku ≈ US$ 1/1M entrada e US$ 5/1M saída):

O QUÊ	CONTA	CUSTO
Indexar as 10 lembranças (~5k tokens de embeddings)	5k × Titan	**~US$ 0,0001**
Uma pergunta (embedding + ~1,5k in + ~300 out no Haiku)		**~US$ 0,003**
A festa inteira (200 perguntas da família)	200 × acima	**~US$ 0,60**
EC2 t3.micro, mês inteiro	730 h	US$ 7,60 (ou Free Tier)
S3 (lembranças + site)	< 1 GB	< US$ 0,10

Frase de efeito do material (verdadeira): **o desafio inteiro custa menos que um pão de queijo**. E o item caro da arquitetura nem é a IA, é a máquina ociosa. Gancho para aula futura: "por que isso um dia vira Lambda".

09 O ponto de arremate (tarefa 8)

Ativar a flag e provocar o erro de propósito:

```
sudo docker rm -f caixa
sudo docker run -d --name caixa --restart unless-stopped -p 3000:3000 \
  -e S3_BUCKET_LEMBRANCAS=cecilia-lembrancas-SEUNOME \
  -e AWS_REGION=us-east-1 \
  -e SALVAR_INDICE=true \
  ghcr.io/liga-de-computacao-em-nuvem-pucrs/desafio-03-velhinha-backend:latest

sudo docker exec caixa wget -qO- --post-data='' localhost:3000/api/reindexar
sudo docker logs caixa   # → AccessDenied ao salvar: falta s3:PutObject
```

O diff correto na policy é **um statement** (ou uma ação a mais no `LerLembrancas`; melhor um Sid próprio, com o recurso estreito):

```
{
  "Sid": "SalvarIndice",
  "Effect": "Allow",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::cecilia-lembrancas-SEUNOME/_indice/*"
}
```

A decisão que vale nota: o recurso não é `/*`, é só o prefixo `_indice/` onde o backend grava. A API continua **incapaz** de alterar as lembranças da vó; ela só pode escrever o próprio caderno de índice. Least privilege na segunda iteração, que é onde ele costuma morrer no mundo real.

A prova: `sudo docker restart caixa` → nos logs, a linha **"Índice restaurado do bucket: N trecho(s), sem gastar embedding"**, e o boot responde em segundos, sem nova cobrança de embeddings. Economia por boot:

~5k tokens de Titan. Mínima em dinheiro, mas o princípio de não pagar duas vezes pelo mesmo cálculo é o que escala.

10 Encerrando

A limpeza:

1. Instância: Terminate (sem ASG desta vez: ela morre e fica morta).
2. IAM: apagar a role `cecilia-caixa-role` e a policy.
3. Buckets: esvaziar e apagar os dois (o de lembranças tem o `_indice/` do arremate dentro; o "esvaziar" do console pega tudo).
4. Model access pode ficar habilitado: parado, custa zero.
5. Billing no dia seguinte.

SINTOMA	CAUSA QUASE SEMPRE
`bucket: "sem_acesso"` no /health	role não anexada, ou ListBucket/GetObject com os ARNs trocados
`CredentialsProviderError` / timeout do SDK	hop limit 1 (Modify instance metadata options → 2 → restart do container)
Erro mandando habilitar modelo	Model access pendente, ou habilitado em outra região
`on-demand throughput isn't supported`	MODELO_CHAT sem o prefixo `us.`
`AccessDenied` só no /api/perguntar	policy cobre o profile mas não os foundation models `us-*` (decisão 2 da seção 04)
Vó não lembra de nada	`documentosIndexados: 0` : bucket vazio, extensão errada, ou faltou reindexar
Vó responde em inglês / seca	MODELO_CHAT trocado por engano; o prompt de persona está em `rag.js`, conferir o modelo
Site sem dados	config.js sem a porta 3000, cache, SG sem a 3000, ou site aberto em https

O que separa o grupo que tira nota cheia: a policy com três ações e ARNs certos de primeira; a explicação profile → foundation models `us-*` sem consultar; a recusa honesta demonstrada ao vivo; e o arremate com o `PutObject` restrito ao prefixo `_indice/` .